

BOOK 15A (SPECIAL) — JAVA FULL STACK DEVELOPER

Frontend & Integration: HTML/CSS/JS/TS/React + Spring Boot (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 15A of 24

(Special/Supplementary — inserted after Book 15, before Book 16) **Why this book exists:** Recruiter-search data for this profile shows **Spring Boot** as the single most-searched keyword, followed by **Java, J2EE, Java+Spring Boot, Microservices**, with "**Java Full Stack Developer**" as the targeted role (3–5 years experience band). This book exists specifically to make the "**Full Stack**" half of that title genuinely true — the Java/Spring Boot depth is already being built in Books 09–15 and continues in Books 16–24; this book adds the frontend layer and, more importantly, the **integration** patterns that Full Stack interviews actually probe. **Versions Covered:** HTML5, CSS3, ES2022+ JavaScript, TypeScript 5.x, React 18 **Prerequisites:** Book 10 (HTTP/REST/JSON/CORS), Book 12 (Spring Boot REST APIs), Book 14 (JWT authentication) — this book is the frontend counterpart to those backend chapters **Fits Between:** 15_Java_Testing.md and 16_Microservices.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: "Java Full Stack Developer" job posting లు, JD లో దాదాపు ఎప్పుడూ React/Angular + REST integration అడుగుతాయి, పూర్తి frontend framework mastery కాదు. ఈ పుస్తకం యొక్క goal: మీరు React తో ఒక UI build చేయగలగాలి, దాన్ని Book 12 లో మీరు build చేసిన Spring Boot API కి connect చేయగలగాలి, Book 14 యొక్క JWT auth ని frontend లో సరిగ్గా handle చేయగలగాలి, మరియు ఈ మొత్తం flow ని interview లో confidently వివరించగలగాలి.

English: "Java Full Stack Developer" job postings almost always test React/Angular + REST integration knowledge, not deep frontend framework mastery. This book's goal: you should be able to build a UI in React, connect it to the Spring Boot API you built in Book 12, correctly handle Book 14's JWT auth on the frontend, and confidently explain that entire flow in an interview.

Learning Objectives

1. Write correct, semantic HTML5 and responsive CSS3.
2. Master modern JavaScript (ES6+) — the language every frontend framework sits on.
3. Understand TypeScript's value proposition and use it confidently.
4. Build functional React components with hooks and manage component state.
5. Connect a React frontend to a Spring Boot REST API correctly (fetch/axios, CORS, error handling).
6. Implement JWT-based authentication end-to-end, from React login form to Spring Security backend.

7. Understand state management approaches (Context API, and when Redux-style tools are warranted).
 8. Explain full-stack architecture and deployment concerns in interviews.
-



Table of Contents

Ch.	Title
1	The Full Stack Landscape & Why This Book Exists
2	HTML5 Fundamentals for Full Stack Developers
3	CSS3 & Responsive Design
4	JavaScript (ES6+) Fundamentals
5	TypeScript Fundamentals
6	React Fundamentals: Components, Props, Hooks
7	Connecting React to a Spring Boot REST API
8	Frontend Authentication: JWT End-to-End
9	State Management
10	Full Stack Architecture & Deployment Overview
11	Mini Project — Full Stack Task Manager
—	Final Revision Notes, Cheat Sheet, Full Stack Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — The Full Stack Landscape & Why This Book Exists

Telugu Explanation

"Java Full Stack Developer" role లో, recruiter mainly Java/Spring Boot depth చూస్తారు (మీ recruiter-search data ఇదే చూపిస్తుంది — Spring Boot 4 సార్లు, Java 2 సార్లు, Microservices 2 సార్లు), కానీ JD లో "React/Angular" ఒక **checkbox requirement** గా ఉంటుంది — meaning, మీరు ఒక functional React app build చేయగలగాలి, backend కి connect చేయగలగాలి, కానీ React internals లో PhD స్థాయి depth expect చేయరు. ఈ పుస్తకం సరిగ్గా ఆ స్థాయి కి target చేయబడింది — Java/Spring Boot depth (ఇతర books లో) + frontend competence + integration fluency.

Professional English Explanation

In a "Java Full Stack Developer" role, recruiters primarily screen for Java/Spring Boot depth (exactly what your recruiter-search data shows — Spring Boot appearing most frequently, followed by Java, Microservices), while "React/Angular" in the JD functions more as a **checkbox competency** — you need to build a functional React app and connect it to a backend, not demonstrate framework-internals-level expertise. This book targets exactly that level: real frontend competence and integration fluency, layered on top of the deep Java/Spring Boot foundation the rest of this series already builds.

The Recruiter-Aligned Learning Hierarchy (This Book's Place In It)

```
Java Foundation (Books 01-08)
|
v
Advanced Java / J2EE concepts (Book 10)
|
v
★ Spring Framework (Book 11)
|
v
★★★★ Spring Boot (Book 12)
|
v
★★★★ Spring Boot + REST + JPA/Hibernate + Security (Books 12-14)
|
v
[THIS BOOK: Frontend + Integration – makes "Full Stack" real]
|
v
★★★★ Microservices (Book 16)
|
v
Kafka + Redis + Docker + Kubernetes + Cloud + CI/CD (Books 17, 21-24)
|
v
★ Java Full Stack – the complete, interview-ready profile
|
v
Real-world projects + production architecture + interviews (Books 22-24)
```

Real-World Example

Telugu: Real "Java Full Stack Developer" interviews లో, 70-80% questions Java/Spring Boot/Microservices మీద ఉంటాయి (ఇది మీ core books లో పూర్తిగా cover అవుతుంది), 15-20% React/frontend integration మీద (ఇది ఈ పుస్తకం cover చేస్తుంది), మరియు 5-10% general full-stack architecture మీద (deployment, CI/CD — Book 16+ లో). English: Real "Java Full Stack Developer" interviews typically spend 70-80% of questions on Java/Spring Boot/Microservices (fully covered by your core books), 15-20% on React/frontend integration (this book's scope), and 5-10% on general full-stack architecture (deployment, CI/CD — Books 16+).

Interview Answer

"As a Java Full Stack Developer, my primary depth is in Java, Spring Boot, and Microservices — building REST APIs, securing them with Spring Security/JWT, and persisting data with Spring Data JPA. On the frontend, I'm comfortable building functional React applications that consume those APIs, handle authentication correctly, and manage application state — enough to own a feature end-to-end without needing a separate frontend specialist for straightforward CRUD-and-auth workflows."

Mastery Check

Without notes, explain in one paragraph why a Java Full Stack Developer's interview preparation should weight Java/Spring Boot/Microservices far more heavily than frontend framework depth, using your own recruiter-search data as the justification.

CHAPTER 2 — HTML5 Fundamentals for Full Stack Developers

Telugu Explanation

HTML5 semantic elements (`<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<footer>`) page structure ని meaningful గా చేస్తాయి — accessibility మరియు SEO కి కూడా ముఖ్యం. Forms (`<form>` , `<input>` , validation attributes) backend కి data పంపడానికి foundation.

Professional English Explanation

HTML5 semantic elements (`<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<footer>`) give page structure real meaning — important for accessibility and SEO, not just visual layout. Forms (`<form>` , `<input>` , validation attributes) are the foundation for sending data to a backend.

Java Code — An HTML Form That Will Feed a Spring Boot API (Ch.7)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Manager</title>
</head>
<body>
  <header>
    <h1>Task Manager</h1>
    <nav>
      <a href="/tasks">Tasks</a>
      <a href="/login">Login</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Create a Task</h2>
      <!-- This form's fields map DIRECTLY to Book 12, Ch.4's CreateTaskRequest DTO -->
      <form id="taskForm">
        <label for="title">Title</label>
        <input type="text" id="title" name="title" required minlength="1" maxlength="200">

        <label for="priority">Priority</label>
        <input type="number" id="priority" name="priority" min="1" max="5" required>

        <button type="submit">Create Task</button>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Task Manager</p>
  </footer>
</body>
</html>
```

Internal Working

- The `required`, `minlength`, `maxlength`, `min`, `max` attributes provide **client-side validation** — a genuine UX improvement (immediate feedback, no round-trip needed) but **never a substitute** for server-side validation (Book 12, Ch.5's `@Valid` / Bean Validation) — client-side validation can always be bypassed (disabled JavaScript, a direct API call via `curl` / Postman), so the server must independently enforce every rule regardless of what the HTML claims to check.
- Semantic elements (`<nav>`, `<main>`, `<section>`) don't change visual rendering by themselves (that's CSS's job, Ch.3) but give assistive technology (screen readers) and search engines structural meaning — a real, professional consideration beyond "just making it look right."

Real-World Example

Telugu: Real production forms — client-side validation (`required`, `pattern`) UX కోసం, కానీ backend Bean Validation (Book 12, Ch.5) ఖచ్చితంగా ప్రతి field ని independently verify చేస్తుంది, ఎందుకంటే client-side checks bypass చేయబడొచ్చు. English: Real production forms use client-side validation for immediate UX feedback, but the backend's Bean Validation (Book 12, Ch.5) independently verifies every field regardless, since client-side checks can always be bypassed.

Interview Answer

"HTML5 semantic elements structure a page meaningfully for accessibility/SEO, and forms provide the client-side data-entry mechanism that maps to backend DTOs. Client-side validation attributes improve UX but are never a substitute for server-side validation — they can be bypassed, so the Spring Boot backend (Book 12, Ch.5) must always independently enforce every business rule."

Coding Exercise

L1: Build a semantic HTML page with header/nav/main/footer for a task list UI. **L2:** Build a form matching Book 12's `CreateTaskRequest` DTO fields exactly, with appropriate client-side validation attributes. **L4 (Interview):** Explain why client-side validation is never sufficient on its own. **L6 (Mastery):** Explain, from memory, why HTML form field names should match backend DTO field names deliberately.

CHAPTER 3 — CSS3 & Responsive Design

Telugu Explanation

CSS **Flexbox** మరియు **Grid** modern layout కి standard tools. **Responsive design** (@media queries) ఒకే HTML, వేర్వేరు screen sizes (mobile, tablet, desktop) కి సరిగ్గా render అవ్వడానికి. Full Stack interviews లో CSS deep expertise expect చేయరు, కానీ basic layout+responsiveness build చేయగలగాలి.

Professional English Explanation

CSS **Flexbox** and **Grid** are the standard modern layout tools. **Responsive design** (@media queries) lets one HTML structure render correctly across screen sizes (mobile, tablet, desktop). Full Stack interviews don't expect deep CSS expertise, but you should be able to build basic layout and responsiveness.

Java Code — CSS for the Task Manager

```
.task-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
  max-width: 600px;
  margin: 0 auto;
}

.task-card {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.task-card.completed {
  opacity: 0.6;
  text-decoration: line-through;
}

/* Responsive: stack differently on small screens */
@media (max-width: 600px) {
  .task-card {
    flex-direction: column;
    align-items: flex-start;
  }
}
```

Internal Working

- Flexbox's `justify-content` / `align-items` control alignment along the main/cross axis of a one-dimensional layout; CSS Grid (not shown here but worth knowing conceptually) handles genuinely two-dimensional layouts (rows AND columns together) — Flexbox is typically sufficient

and simpler for component-level layouts like a task card, which is why it's the more commonly reached-for tool in day-to-day full-stack work.

- `@media (max-width: 600px)` is a **breakpoint** — CSS conditionally applies rules based on the viewport width, letting one HTML/CSS codebase serve both desktop and mobile without separate pages — the responsive design principle underlying virtually all modern web UIs.

Interview Answer

"Flexbox handles one-dimensional component layout (a row of items, a column of cards); Grid handles two-dimensional layouts. Responsive design via `@media` queries lets one codebase adapt to different screen sizes via breakpoints. For a Java Full Stack role, the expectation is functional, clean layout — not deep CSS architecture expertise."

Coding Exercise

L1: Style the task list from Ch.2's HTML using Flexbox, including a responsive breakpoint for mobile.

L4 (Interview): Explain the difference between Flexbox and Grid, and when you'd choose each.

CHAPTER 4 — JavaScript (ES6+) Fundamentals

Telugu Explanation

Modern JavaScript (ES6+) React/Angular రెండింటికీ foundation. ముఖ్యమైన concepts: `let / const` (Java's `final` కి similar), **arrow functions**, **destructuring**, **template literals**, **Promises/async-await** (Book 07's `CompletableFuture` కి JavaScript-side equivalent — asynchronous API calls handle చేయడానికి), **modules** (`import / export`).

Professional English Explanation

Modern JavaScript (ES6+) underlies both React and Angular. Key concepts: `let / const` (similar to Java's `final`), **arrow functions**, **destructuring**, **template literals**, **Promises/async-await** (the JavaScript-side equivalent of Book 07's `CompletableFuture` — for handling asynchronous API calls), and **modules** (`import / export`).

Java Code — JavaScript Fetching From Your Spring Boot API

```
// const/let (like Java's final vs regular variables, Book 01 Ch.9)
const API_BASE_URL = "http://localhost:8080/api"; // never reassigned
let taskCount = 0;                               // can be reassigned

// Arrow functions (concise, like Java lambdas, Book 07 Ch.2)
const double = (x) => x * 2;

// Destructuring - extract fields directly, like unpacking a Java record (Book 07, Ch.13)
const task = { id: 1, title: "Buy groceries", completed: false };
const { title, completed } = task;

// Template literals - like Java's text blocks (Book 07, Ch.14), string interpolation
console.log(`Task "${title}" is ${completed ? "done" : "pending"}`);

// ASYNC/AWAIT: the JavaScript equivalent of Book 07's CompletableFuture chaining
async function fetchTasks() {
    try {
        const response = await fetch(`${API_BASE_URL}/tasks`); // like Java 11's HttpClient
        // (Book 10, Ch.1)
        if (!response.ok) {
            throw new Error(`HTTP error: ${response.status}`); // Book 10, Ch.1's status
            // codes matter here too
        }
        const tasks = await response.json(); // parses JSON body (Book
        // 10, Ch.6's Jackson, client-side)
        return tasks;
    } catch (error) {
        console.error("Failed to fetch tasks:", error); // NEVER swallow errors
        // silently (Book 04, Ch.7)
        throw error;
    }
}

// Modules - like Java's package/import system (Book 01, Ch.10)
export { fetchTasks, API_BASE_URL };
```

Internal Working

- `async / await` is syntactic sugar over JavaScript Promises (conceptually parallel to Book 07's `CompletableFuture`) — `await` pauses execution of the `async` function (not the whole browser/thread) until the Promise resolves, letting asynchronous code **read like** sequential code while still being non-blocking under the hood — directly analogous to why `CompletableFuture` chaining exists in Java (Book 07, Ch.11).
- `fetch()` resolving successfully does **not** mean the HTTP request succeeded at the application level — `fetch()` only rejects on genuine network failure; a `404` or `500` response still resolves normally, which is exactly why checking `response.ok` (or `response.status`) explicitly is required, mirroring Book 10, Ch.1's status-code discipline on the client side too.

Real-World Example

Telugu: React components (Ch.6) లో API calls ఖచ్చితంగా ఈ `async / await + fetch` పద్ధతిలోనే రాస్తారు — Book 12 లో మీరు build చేసిన REST endpoints ని ఇలాగే consume చేస్తారు. English: React components (Ch.6) write API calls using exactly this `async / await + fetch` pattern — consuming the exact REST endpoints you built in Book 12.

Interview Answer

"Modern JavaScript's `async / await` provides non-blocking asynchronous code that reads sequentially, conceptually parallel to Java's `CompletableFuture` (Book 07). A critical detail: `fetch()` only rejects on network failure, not HTTP error status codes — checking `response.ok` explicitly is required to correctly handle 4xx/5xx responses from a Spring Boot API."

Cross Questions

- Q: Does `fetch()` throw an error for a 404 response? → A: No — it resolves normally; you must check `response.ok / response.status` explicitly to detect HTTP-level errors.
- Q: How does `async / await` relate to Java's `CompletableFuture`? → A: Both provide non-blocking asynchronous composition; `await` lets `async` JavaScript code read sequentially, similar in spirit to how `CompletableFuture` chaining composes `async` Java operations.

Coding Exercise

L1: Write an `async` function fetching from a public test API and correctly handling both success and HTTP-error responses. **L4 (Interview):** Explain why checking `response.ok` is necessary even inside a successful `await fetch()` call.

CHAPTER 5 — TypeScript Fundamentals

Telugu Explanation

TypeScript JavaScript మీద **static typing** add చేస్తుంది (Java లాగే compile-time type checking) — React/Angular production codebases దాదాపు ఎప్పుడూ TypeScript వాడతాయి, ఎందుకంటే large codebases లో type errors compile time లోనే catch అవుతాయి (Book 06's generics/Book 01's static typing philosophy కి direct parallel).

Professional English Explanation

TypeScript adds **static typing** on top of JavaScript (compile-time type checking, just like Java) — production React/Angular codebases almost always use TypeScript, since type errors are caught at compile time in large codebases, a direct parallel to Book 06's generics and Book 01's static-typing philosophy.

Java Code — TypeScript Interfaces Mirroring Your Spring Boot DTOs

```
// Interface mirroring Book 12's TaskResponse DTO EXACTLY - keeps frontend/backend contracts in sync
interface TaskResponse {
  id: number;
  title: string;
  completed: boolean;
  createdAt: string; // JSON dates arrive as strings, then parsed
}

interface CreateTaskRequest {
  title: string;
  priority: number;
}

// Typed function - like a Java method signature (Book 01, Ch.6)
async function createTask(request: CreateTaskRequest): Promise<TaskResponse> {
  const response = await fetch("http://localhost:8080/api/tasks", {
    method: "POST", // Book 10, Ch.1's REST methods
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(request),
  });
  if (!response.ok) throw new Error(`Failed to create task: ${response.status}`);
  return response.json() as Promise<TaskResponse>;
}

// Compile-time error prevention - TypeScript catches this BEFORE running, like Java's compiler (Book 01, Ch.1)
// createTask({ title: "Test" }); // ERROR: Property 'priority' is missing - caught at COMPILE time, not runtime!
```

Internal Working

- TypeScript's `interface` declarations serve the same conceptual role as Java's DTO classes/records (Book 12, Ch.4) — they define a **contract** the compiler enforces; the

commented-out error above demonstrates exactly the kind of "missing required field" bug that Java's compiler (Book 01, Ch.1) and Book 12, Ch.5's `@Valid` would also catch, just at a different point (TypeScript catches it at compile time on the frontend; Bean Validation catches it at request time on the backend) — professional full-stack teams deliberately keep these two type definitions in sync.

- TypeScript compiles down to plain JavaScript (there's no TypeScript runtime) — exactly analogous to Book 01, Ch.1's `javac` compiling Java source to bytecode; type checking is a **compile-time-only** discipline (like Book 06's generics via erasure), providing zero runtime overhead or protection once compiled.

Real-World Example

Telugu: Real full-stack teams TypeScript interfaces ని backend DTOs తో manual గా (లేదా code-generation tools తో) sync గా ఉంచుతారు — frontend/backend contract mismatch వల్ల production bugs రాకుండా. English: Real full-stack teams keep TypeScript interfaces manually (or via code-generation tools) in sync with backend DTOs — preventing frontend/backend contract mismatches from becoming production bugs.

Interview Answer

"TypeScript adds compile-time static typing to JavaScript, exactly parallel to Java's own compile-time type checking. TypeScript interfaces mirror backend DTOs, letting the frontend compiler catch contract mismatches (missing fields, wrong types) before the code ever runs — the same category of safety Book 12's `@Valid` provides on the backend, just enforced at a different point in the pipeline."

Coding Exercise

L1: Write TypeScript interfaces mirroring 2 of your Book 12 DTOs exactly. **L4 (Interview):** Explain why TypeScript interfaces and backend DTOs should be kept in sync, and what risk exists if they drift apart.

CHAPTER 6 — React Fundamentals: Components, Props, Hooks

Telugu Explanation

React UI ని **components** గా build చేస్తుంది — reusable, self-contained pieces (Book 02's class/object concept కి UI-level parallel). **Props** parent నుండి child కి data పంపడానికి (method parameters లాగే).

useState hook component తనదైన state maintain చేయడానికి (Java object యొక్క instance field లాగే, కానీ change అయినప్పుడు UI automatic గా re-render అవుతుంది). **useEffect** side effects (API calls) కి.

Professional English Explanation

React builds UIs as **components** — reusable, self-contained pieces (a UI-level parallel to Book 02's class/object concept). **Props** pass data from parent to child (like method parameters). The **useState** hook lets a component maintain its own state (like an instance field, but changing it automatically triggers UI re-render). **useEffect** handles side effects (like API calls).

Java Code — A React Component Consuming Your Spring Boot API

```
import { useState, useEffect } from "react";

function TaskList() {
  const [tasks, setTasks] = useState([]);           // like a private instance field (Book 02, Ch.2) + auto re-render
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {                                // runs after render - like Book 11's
    @PostConstruct, but per-render
    fetch("http://localhost:8080/api/tasks")
      .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then((data) => setTasks(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);                                           // empty dependency array = run ONCE,
  like a component's "init"

  if (loading) return <p>Loading tasks...</p>;
  if (error) return <p>Error: {error}</p>;           // Book 04's exception-handling
  philosophy, UI-side

  return (
    <div className="task-list">                      /* Ch.3's CSS class */
      {tasks.map((task) => (
        <TaskCard key={task.id} task={task} />       // 'key' - React needs a stable
        identity per list item
      ))}
    </div>
  );
}

function TaskCard({ task }) {                       // PROPS - data passed from parent
  (TaskList) to child
  return (
    <div className={`task-card ${task.completed ? "completed" : ""}`>
      <span>{task.title}</span>
      <span>Priority: {task.priority}</span>
    </div>
  );
}

export default TaskList;
```

Internal Working

- `useState` 's critical behavior: calling `setTasks(data)` doesn't just update a variable — it triggers React to **re-render** the component (and its children) with the new state, exactly the reactive-UI paradigm that distinguishes React from manually mutating the DOM; this is conceptually similar to Book 13, Ch.3's dirty checking (a state change is automatically detected and something happens as a consequence) but at the UI-rendering layer instead of the database layer.

- The `useEffect` dependency array (`[]` here) controls **when** the effect re-runs — an empty array means "run once, after the first render" (the closest UI analog to a one-time initialization), while omitting it entirely would re-run the effect after **every** render (almost always a bug for an API call, causing an infinite fetch loop) — this is a genuinely common, frequently-tested React gotcha.
- The `key` prop on list items isn't just a formality — React uses it to efficiently determine which list items changed/were added/removed between re-renders, avoiding unnecessary re-creation of unchanged DOM elements; using an unstable key (like the array index, when items can be reordered) is a real, documented React anti-pattern.

Real-World Example

Telugu: ఈ `TaskList` component ఖచ్చితంగా Book 12లో మీరు build చేసిన `GET /api/tasks` endpoint ని consume చేస్తుంది — full-stack interview లో "React ని backend కి ఎలా connect చేస్తారు" అనే question కి ఇదే expected సమాధానం shape. English: This `TaskList` component consumes exactly the `GET /api/tasks` endpoint you built in Book 12 — the expected answer shape for a full-stack interview's "how do you connect React to a backend" question.

Interview Answer

"React components are reusable UI pieces; props pass data down from parent to child; `useState` gives a component reactive local state — updating it triggers an automatic re-render, conceptually similar to how a change in managed data triggers a UI update. `useEffect` with an empty dependency array runs once after the initial render, the standard pattern for an initial API fetch — omitting the dependency array entirely is a common bug causing an infinite re-fetch loop."

Cross Questions

- Q: What happens if you omit the dependency array in `useEffect` ? → A: The effect re-runs after every render — for an API call, this typically causes an infinite fetch loop, since each fetch's state update triggers a re-render, which re-runs the effect again.
- Q: Why does React need a stable `key` for list items? → A: To efficiently determine which items changed between renders, avoiding unnecessary DOM recreation — an unstable key (like array index on a reorderable list) can cause real rendering bugs.

Coding Exercise

L1: Build a `TaskList` component fetching from a real or mocked API, handling loading and error states. **L3:** Reproduce the "missing dependency array" infinite-loop bug, then fix it. **L4 (Interview):** Explain `useState` and `useEffect`, and the empty-dependency-array pattern. **L6 (Mastery):** Explain, from memory, why calling `setState` triggers a re-render, connecting it conceptually to Book 13's dirty checking as a "state change triggers automatic consequence" pattern.

CHAPTER 7 — Connecting React to a Spring Boot REST API

Telugu Explanation

ఇది ఈ పుస్తకం యొక్క **most interview-critical chapter** — React frontend, Spring Boot backend కి ఎలా correctly connect అవుతుందో (CORS, Book 10 Ch.7's Bearer header, error handling, environment-specific API URLs).

Professional English Explanation

This is the **most interview-critical chapter** in this book — exactly how a React frontend correctly connects to a Spring Boot backend (CORS, Book 10 Ch.7's Bearer header pattern, error handling, environment-specific API URLs).

Java Code — Full Integration Layer

```
// api.js - a centralized API client (like Book 09, Ch.10's DAO pattern, but for frontend HTTP calls)
const API_BASE_URL = process.env.REACT_APP_API_URL || "http://localhost:8080/api"; // Book 12, Ch.8's profile-driven config, frontend side

async function apiRequest(endpoint, options = {}) {
  const token = localStorage.getItem("accessToken"); // Ch.8's JWT storage

  const response = await fetch(`${API_BASE_URL}${endpoint}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(token && { Authorization: `Bearer ${token}` }), // Book 10, Ch.7 / Book 14, Ch.4's Bearer pattern
    },
  });

  if (response.status === 401) {
    localStorage.removeItem("accessToken"); // token invalid/expired - force re-login
    window.location.href = "/login";
    throw new Error("Session expired");
  }

  if (!response.ok) {
    const errorBody = await response.json().catch(() => ({})); // Book 12, Ch.6's ErrorResponse shape
    throw new Error(errorBody.message || `Request failed: ${response.status}`);
  }

  if (response.status === 204) return null; // Book 10, Ch.1's "No Content" - nothing to parse
  return response.json();
}

export const TaskApi = {
  getAll: () => apiRequest("/tasks"),
  getById: (id) => apiRequest(`/tasks/${id}`),
  create: (task) => apiRequest("/tasks", { method: "POST", body: JSON.stringify(task) }),
  update: (id, task) => apiRequest(`/tasks/${id}`, { method: "PUT", body: JSON.stringify(task) }),
  delete: (id) => apiRequest(`/tasks/${id}`, { method: "DELETE" }),
};
```

```
// REMINDER: the Spring Boot side (Book 14, Ch.1) MUST allow this frontend's origin
@Bean
@ConfigurationSource corsConfigurationSource() {
  CorsConfiguration config = new CorsConfiguration();
  config.setAllowedOrigins(List.of("http://localhost:3000")); // React dev server's default port
  config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
  config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
  // ... (full config in Book 14, Ch.7)
}
```

Internal Working

- Centralizing the `fetch` logic in one `apiRequest()` function (rather than repeating `fetch/header/error-handling` boilerplate in every component) is the frontend equivalent of Book 09, Ch.10's DAO pattern — one place owns cross-cutting concerns (auth header attachment, 401 handling, error parsing), and every API call benefits automatically, instead of that logic being duplicated and potentially inconsistently maintained across dozens of components.
- The 401 handling here (clearing the stored token and redirecting to login) is the frontend counterpart to Book 14, Ch.4's backend JWT validation — when the backend's `JwtAuthenticationFilter` rejects an expired/invalid token, the backend correctly returns 401 (Book 10, Ch.1), and the frontend's job is to recognize that specific status and respond appropriately (force re-authentication), rather than showing a confusing generic error.
- `process.env.REACT_APP_API_URL` mirrors exactly Book 12, Ch.8's externalized configuration philosophy — the API's base URL differs between local development, staging, and production, and hardcoding it would break the "same artifact, different environments" principle Book 12, Ch.13 established for the backend.

Real-World Example

Telugu: ఈ `TaskApi` object ఖచ్చితంగా real production React apps లో "API layer" నా ఉంటుంది — Book 12-14 లో మీరు build చేసిన controller/security stack తో నేరుగా matching, full-stack interview లో "మీరు frontend/backend ఎలా integrate చేస్తారు" అనే question కి ఇదే expected, complete సమాధానం. English: This `TaskApi` object is exactly the shape of a real production React app's "API layer" — directly matching the controller/security stack built in Books 12-14, and exactly the complete, expected answer to a full-stack interview's "how do you integrate frontend and backend" question.

Interview Answer

"Frontend-backend integration requires: a centralized API client (mirroring the DAO pattern, Book 09) attaching the JWT Bearer token to every request, correctly handling 401 by clearing the stored token and forcing re-login, parsing the backend's structured error response (Book 12, Ch.6) for meaningful error messages, and using environment-specific configuration for the API base URL (mirroring Book 12, Ch.8's profiles). On the backend side, CORS (Book 14, Ch.7) must explicitly allow the frontend's origin, or none of this works regardless of how correct the frontend code is."

Cross Questions

- Q: Why centralize the `fetch` logic in one function instead of calling `fetch()` directly in every component? → A: Cross-cutting concerns (auth header, 401 handling, error parsing) live in one place, applying consistently everywhere, exactly like Book 09, Ch.10's DAO pattern centralizes data-access concerns.
- Q: What should the frontend do when it receives a 401 response? → A: Clear the stored (now-invalid) token and redirect to login, since a 401 specifically means the current authentication is no longer valid (Book 10, Ch.1; Book 14, Ch.4).
- Q: Why must the backend's CORS configuration explicitly list the frontend's origin? → A: Without it, the browser blocks the frontend's JavaScript from reading the API's response entirely (Book

14, Ch.7) — no amount of correct frontend code can work around a missing CORS allowance.

Coding Exercise

L1: Build the centralized `apiRequest()` function and use it for at least 3 different endpoints. **L3:** Reproduce a CORS error by omitting the frontend's origin from the backend's configuration, then fix it. **L5 (Senior):** Design the full error-handling strategy for a production React app's API layer, covering network failures, 4xx, and 5xx responses distinctly. **L6 (Mastery):** Explain, from memory, the complete request path from a React component's API call to a Spring Boot controller and back, naming every mechanism involved (CORS, JWT filter, controller, DTO serialization).

CHAPTER 8 — Frontend Authentication: JWT End-to-End

Telugu Explanation

Book 14లో backend JWT issue/validate చేయడం నేర్చుకున్నాము. ఇప్పుడు frontend side: login form → token receive → token store (localStorage, తో దాని security trade-offs) → protected routes → logout.

Professional English Explanation

Book 14 taught backend JWT issuance/validation. Now, the frontend side: login form → receiving the token → storing it (localStorage, with its security trade-offs) → protected routes → logout.

Java Code

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault(); // stop the browser's default
    // full-page form submission
    try {
      const response = await fetch("http://localhost:8080/api/auth/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username, password }), // matches Book 14, Ch.4's
        // LoginRequest record
      });
      if (!response.ok) throw new Error("Invalid credentials"); // GENERIC message - Book
      // 14, Ch.9's username-enumeration lesson

      const { accessToken } = await response.json();
      localStorage.setItem("accessToken", accessToken); // stored for Ch.7's
      // apiRequest() to attach later
      navigate("/tasks");
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={username} onChange={(e) => setUsername(e.target.value)}
        placeholder="Username" />
      <input type="password" value={password} onChange={(e) => setPassword(e.target.value)}
        placeholder="Password" />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Login</button>
    </form>
  );
}

// Protected route wrapper - checks for a token BEFORE rendering the protected content
function ProtectedRoute({ children }) {
  const token = localStorage.getItem("accessToken");
  if (!token) {
    window.location.href = "/login"; // no token - redirect BEFORE
    // rendering anything sensitive
    return null;
  }
  return children;
}

function logout() {
  localStorage.removeItem("accessToken"); // Book 14, Ch.5's client-
  // side half of logout
}
```

```
window.location.href = "/login";  
}
```

Internal Working

- Storing the JWT in `localStorage` is the simplest approach and works fine for many applications, but it carries a genuine, known trade-off directly connecting to Book 14, Ch.8: `localStorage` is accessible to **any** JavaScript running on the page, meaning a successful XSS attack (malicious script injection) can steal the token — the alternative (an `HttpOnly` cookie, Book 10 Ch.3) protects against XSS-based theft but reintroduces CSRF considerations (Book 14, Ch.8) since cookies are automatically attached; **this exact trade-off is a genuinely common, well-regarded full-stack interview question.**
- `ProtectedRoute` performing its check **before** rendering any child content is important — a naive implementation that renders the protected UI first and redirects afterward can briefly flash sensitive content, or worse, allow a component's own `useEffect` (Ch.6) to fire an API call before the redirect completes; checking synchronously, before rendering, avoids this.
- The generic "Invalid credentials" error message directly implements Book 14, Ch.9's username-enumeration lesson on the frontend side — the frontend must not, for example, show a different message for "unknown username" versus "wrong password" even if it somehow had that information, since the backend itself deliberately doesn't distinguish them.

Real-World Example

Telugu: Real full-stack interview లో "JWT ని ఎక్కడ store చేస్తారు, ఎందుకు?" అనేది అత్యంత frequently అడిగే question — `localStorage` (XSS risk) vs `HttpOnly` cookie (CSRF consideration) trade-off ని సరిగ్గా explain చేయగలగడం ఒక్క senior-level signal. English: "Where do you store the JWT, and why?" is an extremely commonly asked real full-stack interview question — correctly explaining the `localStorage` (XSS risk) vs `HttpOnly` cookie (CSRF consideration) trade-off is a genuine senior-level signal.

Interview Answer

"After login, the JWT is stored client-side — commonly in `localStorage` for simplicity, attached to subsequent requests via the `Authorization: Bearer` header (Ch.7). This carries an XSS-theft risk, since any JavaScript on the page can read `localStorage`; the alternative, an `HttpOnly` cookie, protects against that but reintroduces CSRF considerations (Book 14, Ch.8) since cookies are automatically attached by the browser. A protected route checks for a valid token before rendering sensitive content, and logout simply clears the stored token client-side while the backend independently handles refresh-token revocation (Book 14, Ch.5)."

Cross Questions

- Q: What's the security trade-off of storing a JWT in `localStorage` versus an `HttpOnly` cookie? → A: `localStorage` is vulnerable to token theft via XSS (any page JavaScript can read it); an `HttpOnly` cookie prevents that but is automatically attached by the browser, reintroducing CSRF risk (Book 14, Ch.8) unless properly mitigated.

- Q: Why should a ProtectedRoute check for a token before rendering, not after? → A: To avoid briefly rendering sensitive content or firing API calls before a redirect to login completes.
- Q: Should the frontend distinguish "wrong username" from "wrong password" in its own error display? → A: No — this would reintroduce Book 14, Ch.9's username-enumeration risk even if the backend itself is already generic; the frontend should also just show one generic message.

Coding Exercise

L1: Build the login form and protected route wrapper from this chapter, connecting to a real or mocked login endpoint. **L3:** Research and summarize, in your own words, the localStorage-vs-HttpOnly-cookie trade-off for JWT storage. **L5 (Senior):** Design a full frontend token-refresh strategy (Book 14, Ch.5) that transparently renews an expired access token using a refresh token, without disrupting the user's current action. **L6 (Mastery):** Explain, from memory, the complete login-to-protected-page flow, naming every storage/redirect/header mechanism involved.

CHAPTER 9 — State Management

Telugu Explanation

చిన్న apps కి `useState` (Ch.6) సరిపోతుంది, కానీ growing app లో "prop drilling" (data ని deep nested components ద్వారా manually pass చేయడం) సమస్య అవుతుంది. **Context API** (React built-in) global-ish state ని (ఉదా. logged-in user) ఎక్కడైనా access చేయనిస్తుంది, prop drilling లేకుండా. Redux/Zustand వంటి libraries పెద్ద, complex apps కి వాడతారు — Full Stack interview కి Context API అర్థం చేసుకుంటే సరిపోతుంది, Redux internals expect చేయరు usually.

Professional English Explanation

`useState` (Ch.6) suffices for small apps, but growing apps run into "prop drilling" (manually passing data through deeply nested components). React's built-in **Context API** lets somewhat-global state (like the logged-in user) be accessed anywhere without prop drilling. Libraries like Redux/Zustand are used for larger, more complex apps — for a Full Stack interview, understanding Context API is usually sufficient; Redux internals aren't typically expected.

Java Code

```
import { createContext, useContext, useState } from "react";

// Context - like a Spring Bean available application-wide via DI (Book 11), but for frontend
state
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(userData, token) {
    localStorage.setItem("accessToken", token);
    setUser(userData);
  }
  function logout() {
    localStorage.removeItem("accessToken");
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

function useAuth() {
  return useContext(AuthContext);
}

// ANY deeply-nested component can now access auth state WITHOUT prop drilling:
function UserBadge() {
  const { user, logout } = useAuth();
  if (!user) return null;
  return (
    <div>
      <span>Welcome, {user.username}</span>
      <button onClick={logout}>Logout</button>
    </div>
  );
}
```

Internal Working

- Context API's `Provider / useContext` pattern is conceptually similar to Book 11's Dependency Injection — a value is made available "application-wide" (within the Provider's subtree) without every intermediate component needing to explicitly receive and re-pass it as a prop, exactly like a Spring bean is injected directly where needed rather than manually threaded through every intervening layer.
- Context is not a full state-management **library** replacement for genuinely complex, high-frequency-update global state — every component consuming a context re-renders whenever that context's value changes, which can become a real performance consideration at scale; this

is exactly why dedicated libraries (Redux, Zustand) exist for larger applications, though for a Full Stack interview and most CRUD-style applications, Context API is the practical, sufficient tool.

Real-World Example

Telugu: Logged-in user information, theme (dark/light mode), language preference — ఇవన్నీ Context API కి classic use cases, ఎందుకంటే app అంతటా ఎక్కడైనా అవసరం అవుతాయి, కానీ ప్రతి component కి prop గా పంపడం impractical. English: Logged-in user info, theme (dark/light mode), and language preference are classic Context API use cases — needed anywhere in the app, but impractical to pass as a prop through every component.

Interview Answer

"Context API solves prop drilling by making state accessible anywhere within a Provider's subtree without manually threading it through every intermediate component — conceptually similar to dependency injection (Book 11). It's sufficient for most CRUD applications' shared state (current user, theme). Dedicated state-management libraries (Redux, Zustand) become relevant for larger apps with complex, high-frequency global state updates, where Context's re-render-on-any-change behavior becomes a real performance concern."

Coding Exercise

L1: Build an `AuthProvider` and `useAuth` hook, and consume it in 2 different nested components without prop drilling. **L4 (Interview):** Explain the prop-drilling problem Context API solves, and its own limitation at scale.

CHAPTER 10 — Full Stack Architecture & Deployment Overview

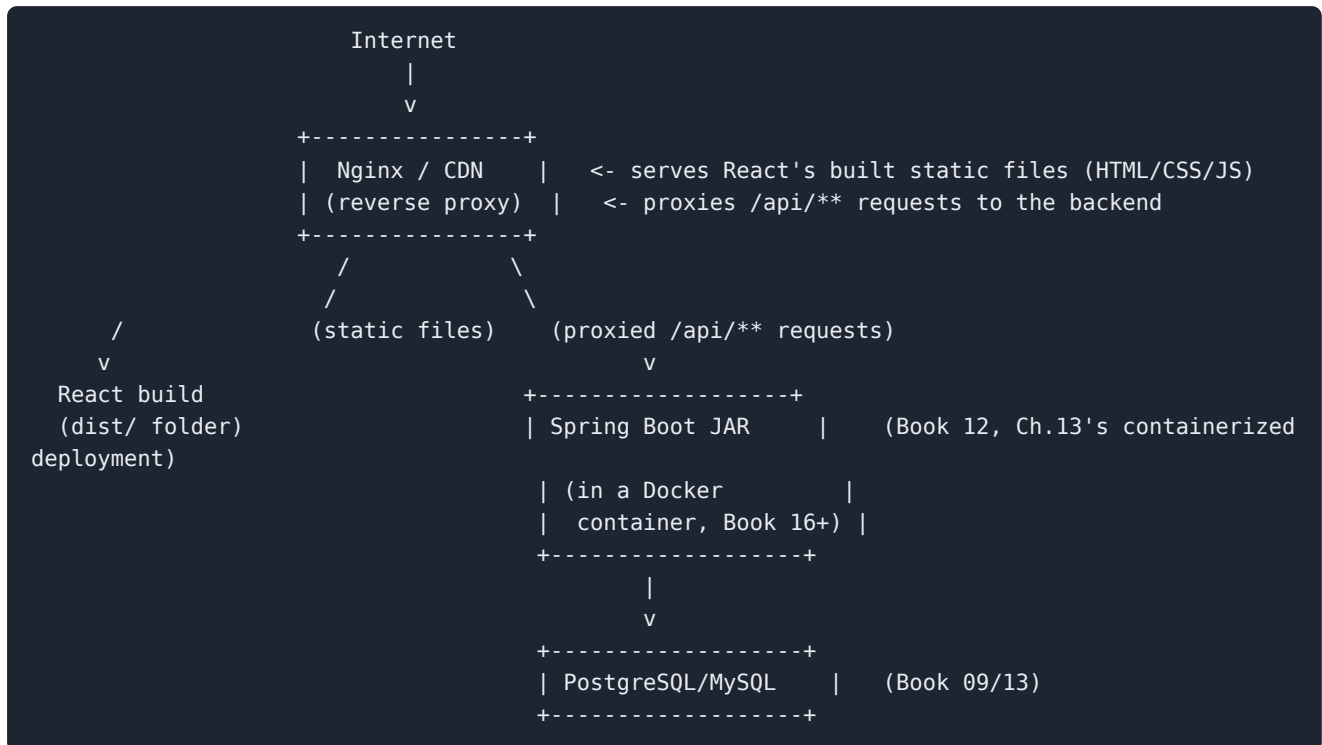
Telugu Explanation

Real full-stack applications frontend (React, static files నా build అవుతుంది) మరియు backend (Spring Boot, Book 12, Ch.13's fat JAR) వేర్వేరుగా deploy అవుతాయి — frontend ఒక static file server/CDN మీద, backend containerized (Docker, Book 16+) నా. Development లో CORS (Ch.7) అవసరం, production లో తరచుగా reverse proxy (Nginx) రెండింటినీ ఒకే domain వెనుక పెడుతుంది.

Professional English Explanation

Real full-stack applications deploy the frontend (React, built to static files) and backend (Spring Boot, Book 12, Ch.13's fat JAR) separately — the frontend on a static file server/CDN, the backend containerized (Docker, Book 16+). CORS (Ch.7) is needed in development; production often uses a reverse proxy (Nginx) to serve both behind one domain, sometimes eliminating the need for CORS in production entirely.

Diagram — Full Stack Production Architecture



Internal Working

- Serving the frontend and backend behind **one domain via a reverse proxy** (rather than two separate domains, as in local development) is a common production pattern that can eliminate CORS (Ch.7) entirely — from the browser's perspective, `/api/**` requests are same-origin if Nginx transparently proxies them to the backend, which is a real, practical architectural choice

worth knowing for interviews, distinct from (but related to) the CORS configuration needed in local development where the React dev server and Spring Boot run on different ports.

- The React "build" step (`npm run build`) compiles JSX/TypeScript (Ch.5) into plain, optimized JavaScript/CSS/HTML static files — conceptually similar to Book 12, Ch.13's Maven/Gradle build producing a runnable JAR from Java source; neither React nor TypeScript exist at runtime in production, exactly like TypeScript compiles away entirely (Ch.5).

Real-World Example

Telugu: Real production deployments Docker Compose లేదా Kubernetes (Book 16+) తో, Nginx + Spring Boot container + database container — అన్నీ కలిపి orchestrate అవుతాయి, ఇదే "Full Stack" role యొక్క production-architecture-awareness expectation. English: Real production deployments orchestrate Nginx + a Spring Boot container + a database container together via Docker Compose or Kubernetes (Book 16+) — exactly the production-architecture awareness a "Full Stack" role is expected to demonstrate in interviews.

Interview Answer

"In production, the React frontend builds to static files served by Nginx or a CDN, while the Spring Boot backend runs containerized (Book 12, Ch.13; Book 16). A reverse proxy commonly routes `/api/**` requests to the backend behind the same domain, which can eliminate CORS in production even though it's needed in local development where frontend and backend run on different ports. This full deployment picture — build, containerize, proxy, orchestrate — is exactly what Books 16+ cover in depth for the backend side."

Coding Exercise

L4 (Interview): Explain why a reverse-proxy architecture can eliminate the need for CORS in production even though it's required in local development. **L5 (Senior):** Design the full deployment architecture (frontend, backend, database, proxy) for a new full-stack application, specifying what each piece is responsible for.

CHAPTER 11 — Mini Project: Full Stack Task Manager

Goal

Build a complete, working full-stack application — React frontend + the Spring Boot backend from Books 12-14 — demonstrating genuine Full Stack competence for interviews.

Requirements

1. **Backend** (reuse Books 12-14): the Task Manager REST API with JWT authentication, validation, and global exception handling.
2. **Frontend** (this book): a React app with a login page, a protected task list page, and a create-task form.
3. **Full integration** (Ch.7-8): centralized API client with JWT attachment, 401 handling, and CORS correctly configured.
4. **State management** (Ch.9): an `AuthProvider` context making the logged-in user available throughout the app.
5. **Responsive UI** (Ch.3): the task list and forms should render sensibly on both desktop and mobile widths.
6. **TypeScript** (Ch.5, stretch): convert the API client and component props to TypeScript with interfaces mirroring the backend DTOs exactly.
7. Be able to **explain the entire flow end-to-end** in an interview: from typing a username/password, through the login request, token storage, protected route rendering, an authenticated API call, and the backend's full request-processing pipeline (Books 10-14).

Concepts Reinforced

Every chapter in this book, integrated with Books 12-14's backend work — genuinely demonstrating the "Full Stack" half of "Java Full Stack Developer," aligned directly with the recruiter-search priorities driving this book's creation.

FINAL REVISION NOTES

- **Positioning:** recruiter data prioritizes Spring Boot/Java/Microservices; this book makes "Full Stack" genuinely true without diluting that core focus — frontend depth here is intentionally scoped to interview-relevant competence, not framework mastery.
 - **HTML/CSS:** semantic structure + client-side validation (never a substitute for backend Bean Validation, Book 12 Ch.5); Flexbox/Grid + `@media` queries for responsive layout.
 - **JavaScript:** `async / await` parallels `CompletableFuture` (Book 07); `fetch()` only rejects on network failure — always check `response.ok`.
 - **TypeScript:** compile-time-only typing (like generics' erasure, Book 06); interfaces should mirror backend DTOs to prevent contract drift.
 - **React:** components/props parallel objects/parameters (Book 02); `useState` triggers re-render like a reactive analog to dirty checking (Book 13); `useEffect`'s empty dependency array = run once, omitting it = common infinite-loop bug.
 - **Integration:** centralize API calls (mirrors the DAO pattern, Book 09); handle 401 by clearing token + redirecting; CORS (Book 14, Ch.7) must explicitly allow the frontend origin.
 - **Auth:** `localStorage` (XSS risk) vs `HttpOnly` cookie (CSRF risk) is a genuine, frequently-asked trade-off (Book 14, Ch.8); `ProtectedRoute` checks before rendering.
 - **State management:** Context API solves prop drilling, conceptually parallel to DI (Book 11); sufficient for most CRUD apps.
 - **Architecture:** reverse proxy can eliminate production CORS; frontend builds to static files (parallel to Book 12's JAR build); full picture connects directly to Book 16+'s Docker/Kubernetes.
-



CHEAT SHEET

Client-side validation = UX only, NEVER replaces backend @Valid (Book 12 Ch.5)
async/await = non-blocking, reads sequentially | fetch() rejects ONLY on network failure - check response.ok!
TypeScript = compile-time only (erased at runtime, like generics Book 06) | interfaces should mirror backend DTOs
React: components=objects | props=parameters | useState=reactive field(triggers re-render) | useEffect([])=run once
 MISSING dependency array = infinite loop risk (common bug)
Integration: centralize fetch logic (DAO pattern, Book 09) | attach JWT via
Authorization:Bearer | handle 401=clear+redirect
CORS (Book 14 Ch.7): backend must explicitly allow frontend origin, or NOTHING works regardless of frontend code
JWT storage: localStorage(XSS risk) vs HttpOnly cookie(CSRF risk) - KNOW THIS TRADE-OFF, frequently asked
Context API: solves prop drilling, like DI (Book 11) | re-renders all consumers on change - not for high-freq global state
Production: React builds to static files (like Book 12's JAR build) | reverse proxy can eliminate prod CORS



FULL STACK INTERVIEW QUESTION BANK

Frontend Fundamentals

1. What is the difference between let, const, and var?
2. Explain async/await and how it relates to Promises.
3. What does TypeScript add to JavaScript, and why do production teams use it?
4. What is the difference between props and state in React?
5. Explain useEffect's dependency array and the infinite-loop bug it can cause if misused.

Integration (Most Interview-Relevant) 6. Walk through, end-to-end, how a React app authenticates a user and makes subsequent authenticated API calls to a Spring Boot backend. 7. Where do you store a JWT on the frontend, and what are the security trade-offs? 8. Why does fetch() not throw an error for a 404 or 500 response, and how do you handle that correctly? 9. Explain CORS from the frontend developer's perspective — what error do you see, and how do you fix it? 10. How would you handle an expired access token gracefully in the frontend (connecting to Book 14's refresh tokens)?

Architecture & Full Stack Thinking 11. Explain the difference between a Java Full Stack Developer's expected frontend depth versus backend depth, and how you'd prioritize interview preparation accordingly. 12. Describe the production deployment architecture for a full-stack application (frontend, backend, database, proxy). 13. Explain why a reverse proxy setup can eliminate the need for CORS in production. 14. Design the state management approach for a medium-sized full-stack application's logged-in user and theme preference. 15. How would you structure a centralized API client for a React app calling a dozen different Spring Boot endpoints, and why?

(These questions are intentionally weighted toward integration and architecture — exactly where real Java Full Stack interviews spend their frontend-adjacent time, per this book's Chapter 1 rationale. Deep Java/Spring Boot/Microservices interview depth continues across the rest of this series, especially Book 23's dedicated Interview Master Book.)



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Build the static HTML/CSS task list from Ch.2-3. **L2 — Intermediate:** Add JavaScript/TypeScript API integration (Ch.4-5, 7) against a real or mocked Spring Boot endpoint. **L3 — Advanced:** Build the full React app (Ch.6, 9) with authentication (Ch.8) end-to-end. **L4 — Interview:** Answer all 15 Full Stack Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 11 mini project fully, including the TypeScript stretch goal. **L6 — Mastery:** Explain the complete login-to-authenticated-API-call flow out loud, from memory, naming every mechanism on both frontend and backend.



ONE-DAY REVISION PLAN (≈4 hours)

Time	Focus
0:00–0:30	Ch.1: Recruiter-aligned positioning — know WHY this book is scoped this way
0:30–1:00	Ch.2–3: HTML/CSS — build the static task list UI
1:00–1:30	Ch.4–5: JavaScript/TypeScript fundamentals
1:30–2:15	Ch.6: React — the highest-density chapter for interviews
2:15–2:30	Break
2:30–3:15	Ch.7: Frontend-backend integration — THE most interview-critical chapter
3:15–3:45	Ch.8: JWT frontend flow — memorize the localStorage-vs-cookie trade-off
3:45–4:00	Ch.9–10: State management, architecture — skim, then answer the Interview Bank from memory



FINAL MASTERY CHECKLIST

- ☐ I can explain why this book is scoped toward integration fluency, not frontend framework depth, using my own recruiter data.
- ☐ I can build semantic HTML and responsive CSS for a basic UI.
- ☐ I can write modern JavaScript with async/await and correctly handle fetch() error cases.
- ☐ I can write TypeScript interfaces mirroring backend DTOs.
- ☐ I can build React components with props, useState, and useEffect correctly.
- ☐ I can build a centralized, JWT-aware API client connecting React to Spring Boot.
- ☐ I can implement and explain the full frontend authentication flow, including the storage trade-off.
- ☐ I can explain Context API and when it's sufficient versus when a dedicated state library is warranted.
- ☐ I can explain full-stack production architecture, including the reverse-proxy CORS trade-off.
- ☐ I built the Full Stack Task Manager mini project, integrating with my Books 12-14 backend.
- ☐ I answered the full Full Stack Interview Bank confidently.

You are now ready to continue the core recruiter-priority track: `16_Microservices.md`.